

Opciones para Organizar Comparadores Personalizados

1. Clases Separadas (Recomendado para Clases de Comparadores Simples)

Cada comparador se implementa como una clase independiente. Esto es útil si los comparadores son relativamente simples y no están directamente relacionados.

Ejemplo:

```
class ComparadorPorNombreCompleto implements Comparator<Contacto> {
    @Override
    public int compare(Contacto c1, Contacto c2) {
        return c1.nombreCompleto.compareToIgnoreCase(c2.nombreCompleto);
    }
}

class ComparadorPorCantidadDeTelefonos implements Comparator<Contacto> {
    @Override
    public int compare(Contacto c1, Contacto c2) {
        return Integer.compare(c2.telefonos.size(), c1.telefonos.size());
    }
}
```

2. Clases Internas Estáticas en la Clase Contacto

Si los comparadores están estrechamente relacionados con la lógica de la clase Contacto, pueden declararse como **clases internas estáticas**. Esto agrupa los comparadores dentro de la clase donde se aplican, mejorando la organización y el acceso.

Ejemplo:

```
class Contacto {
    String nombreCompleto;
    // Otros atributos...

    // Comparador para ordenar por nombre
    public static class ComparadorPorNombreCompleto implements
    Comparator<Contacto> {
        @Override
        public int compare(Contacto c1, Contacto c2) {
            return
            c1.nombreCompleto.compareToIgnoreCase(c2.nombreCompleto);
        }
    }

    // Comparador para ordenar por cantidad de teléfonos
    public static class ComparadorPorCantidadDeTelefonos implements
    Comparator<Contacto> {
        @Override
        public int compare(Contacto c1, Contacto c2) {
            return Integer.compare(c2.telefonos.size(),
            c1.telefonos.size());
        }
    }
}
```

3. Clase Única con Comparadores como Constantes (Recomendado para Proyectos Grandes)

En este enfoque, se agrupan todos los comparadores como constantes estáticas dentro de una clase dedicada, por ejemplo, ContactoComparators. Esto es útil para mantener una mejor organización en proyectos grandes.

Ejemplo:

```
import java.util.Comparator;

public class ContactoComparators {
    // Comparador por nombre completo
    public static final Comparator<Contacto> POR_NOMBRE_COMPLETO = new
    Comparator<Contacto>() {
        @Override
        public int compare(Contacto c1, Contacto c2) {
            return
            c1.nombreCompleto.compareToIgnoreCase(c2.nombreCompleto);
        }
    };

    // Comparador por cantidad de teléfonos
    public static final Comparator<Contacto> POR_CANTIDAD_DE_TELEFONOS =
    new Comparator<Contacto>() {
        @Override
        public int compare(Contacto c1, Contacto c2) {
            return Integer.compare(c2.telefonos.size(),
            c1.telefonos.size());
        }
    };

    // Comparador por país de residencia
    public static final Comparator<Contacto> POR_PAIS_RESIDENCIA = new
    Comparator<Contacto>() {
        @Override
        public int compare(Contacto c1, Contacto c2) {
            return
            c1.paisResidencia.compareToIgnoreCase(c2.paisResidencia);
        }
    };
}
```

4. Uso de Lambdas para Comparadores Simples

Para comparadores sencillos, puedes utilizar **expresiones lambda** directamente al momento de la llamada a `Collections.sort()` o cualquier método similar.

```
Collections.sort(contactos, (c1, c2) ->
c1.nombreCompleto.compareToIgnoreCase(c2.nombreCompleto));
Collections.sort(contactos, (c1, c2) ->
Integer.compare(c2.telefonos.size(), c1.telefonos.size()));
```

¿Cuál Es la Mejor Organización?

1. Clases Separadas:

- Ideal si los comparadores tienen lógica compleja o necesitan ser reutilizados ampliamente.
- Facilita la prueba y mantenimiento de cada comparador individualmente.

2. Clases Internas Estáticas:

- Útil si los comparadores están muy ligados a la lógica de la clase.
- Proporciona una manera ordenada de agrupar la lógica relacionada.

3. Clase Única (Constantes):

- Perfecto para proyectos grandes donde necesitas centralizar los comparadores.
- Proporciona una manera clara de acceder a todos los comparadores en un solo lugar.

4. Lambdas:

- Adecuado para comparadores muy simples o usados puntualmente.
- Menos adecuado para lógica compleja o repetitiva.